YAWM AI

A 10-Agent Daily Planner for Ramadan

LangGraph · LangChain · Model Context Protocol (MCP)

Hala Lakhal | February 22, 2026

Motivation

Ramadan is a blessed month — but it is also deeply demanding. As Muslims, we are called to maintain five daily prayers, recite the Quran, perform morning and evening adhkar, observe Suhoor and Iftar, and continue our worldly responsibilities: work, study, family, and personal goals. Managing all of this manually leads to overwhelm, missed intentions, and poor time use.

YAWM AI was built to solve this. It is a fully autonomous, production-grade multi-agent planning system that reads your real tasks from Notion, retrieves your personal preferences from a RAG database, factors in Ramadan-specific constraints (prayer times, sleep cycles, Qailulah nap), and produces a complete conflict-free daily schedule — written directly to Google Calendar, with a visual schedule card and a curated Islamic podcast notification sent by email.

Agent Overview

YAWM AI is composed of **10 specialized agents** orchestrated as a directed acyclic graph (DAG) using LangGraph. Each agent has a single responsibility. They communicate exclusively through a shared typed state (`YawmState`).

Pattern Choice and Justification — Graph Agent Pattern

YAWM AI is built using **LangChain** and **LangGraph** and follows the **Graph Agent Pattern** — agents are nodes in a directed acyclic graph (DAG), connected by typed edges, sharing a single typed state object (`YawmState`).

LangChain provides the building blocks: `ChatOpenAI` for LLM calls, `create_react_agent` for tool-using agents, message types (`HumanMessage`, `SystemMessage`), and the `langchain-mcp-adapters` bridge that wraps MCP server tools into LangChain-compatible tools automatically.

LangGraph provides the orchestration layer: `StateGraph` to define the graph topology, `add_node` / `add_edge` to wire agents together, native parallel fan-out for agents 4–6, and `astream` to stream live execution events to the interface.

Design choice	Why	Alternative rejected
Graph Agent Pattern	Explicit topology, parallelism native	Sequential chain — no fan-out
Parallel fan-out (4, 5, 6)	Run simultaneously, saves ∼30s	Serial execution — too slow
Typed `YawmState`	No silent overwrites between agents	Untyped dicts — hard to debug
Merge on fan-in	`full_schedule` is merged, never replaced	Simple update — loses blocks
Async throughout	Non-blocking, WebSocket streaming works	Sync — freezes pipeline
MCP for all APIs	Agents never call APIs directly	Direct API calls — no isolation

The graph structure guarantees execution order by construction: DayPlanner cannot run until agents 4, 5, and 6 have all written their blocks into state. CalendarAgent cannot run until the full schedule is assembled. No runtime checks are needed — the graph enforces it.

What Each Agent Does

Agent	Responsibility
1 · TaskCollector	Calls Notion MCP, Todoist MCP to fetch all pending tasks. Returns `raw_tasks` as a normalized list with id, title, priority, deadline, and estimated duration. Google Calendar is never read — it is write-only.
2 · Planner	Receives tasks and user context (mood, Ramadan day). Uses LLM to decide energy level, deep-work window (morning/afternoon), and routing configuration stored in `routing_config`.
3 · Supervisor	Validates the routing config and decides the execution strategy for the parallel fan-out agents (4, 5, 6). Acts as a quality gate before scheduling.
4 · SalahGuardian	Calls AlAdhan MCP to fetch real prayer times for today's date and location. Builds fixed prayer blocks with 20-min duration. Updates the prayer streak counter in local storage.
5 · DhikrAgent	Generates morning adhkar (post-Fajr) and evening adhkar (post-Asr) blocks. Uses hardcoded Islamic content — no LLM needed. Returns `dhikr_blocks`.
6 · QuranWird	Reads the local Quran progress tracker. Computes today's Juz assignment and page count based on Ramadan day. Returns `quran_blocks` and khatm percentage.
7 · DayPlanner	The core scheduling intelligence. Receives all fixed blocks (prayers, dhikr, Quran, sleep), flexible tasks from Notion, personal preferences from RAG (ChromaDB), and the sleep window from the sleep calculator. Uses LLM to build a complete conflict-free schedule from 03:00 to 23:59 as `full_schedule`.
8 · CanvaAgent	Renders the `full_schedule` into a visual PNG schedule card using Pillow. Each block is color-coded by type. Saves to `output/daily_schedule.png`.
8.5 · DeenPodcast	Searches YouTube for an Islamic video in English matching the Ramadan day theme and user mood. Finds the best free 35-min slot in the schedule. Sends a Gmail notification with the video link and the schedule card attached.
9 · CalendarAgent	First deletes all today's YAWM-generated Google Calendar events (tagged `Created by YAWM AI`). Then writes the complete `full_schedule` to Google Calendar with correct colors and descriptions.

MCP Servers

All external integrations run as **MCP (Model Context Protocol) servers** — independent subprocesses communicating over stdio JSON-RPC. Agents never call APIs directly; they call MCP tools via `langchain-mcp-adapters`.

MCP Server	External API	Tools exposed
`notion_mcp.py`	Notion API v1	`notion_list_tasks` — fetches all tasks with title, priority, deadline, estimated time
`google_calendar_mcp.py`	Google Calendar API v3	`gcal_list_events`, `gcal_create_event`, `gcal_delete_event`
`aladhan_mcp.py`	AlAdhan REST API	`get_prayer_times` — returns Fajr, Dhuhr, Asr, Maghrib, Isha for any date/location
`todoist_mcp.py`	Todoist REST API v2	`todoist_list_tasks` — fetches today's tasks
`deen_notify_mcp.py`	YouTube Data API + Gmail SMTP	`search_deen_youtube`, `send_gmail_notify`, `send_whatsapp_notify`
`canva_mcp.py`	Local Pillow renderer	`render_schedule_card` — generates PNG from `full_schedule` blocks

RAG layer (ChromaDB): Personal user preferences are stored as vector embeddings using `sentence-transform` DayPlanner queries the store at runtime to retrieve relevant preferences (e.g. hair appointment = 2h, coding = 2–3h blocks, shower in the evening) and injects them into the scheduling prompt.

System Test

Tester: Hala Lakhal **Date:** 22 February 2026 **Ramadan Day:** 5 **Mood:** Energized

Input prompt:

"Energized, need to finish my presentation, plan my day properly"

Pipeline result:

- 18 schedule blocks built across 10 block types
- 17 events written to Google Calendar
- Prayer streak: 18 days
- Quran: Day 5 · Juz 16 complete · 55.5% of khatm
- Deen Podcast: *RAMADAN 2026 IS APPROACHING* by Mufti Menk — scheduled 20:30
- Gmail notification sent with schedule card attached
- RAG preferences respected: coding block 2h, shower in evening

Web Interface — Chat & Pipeline

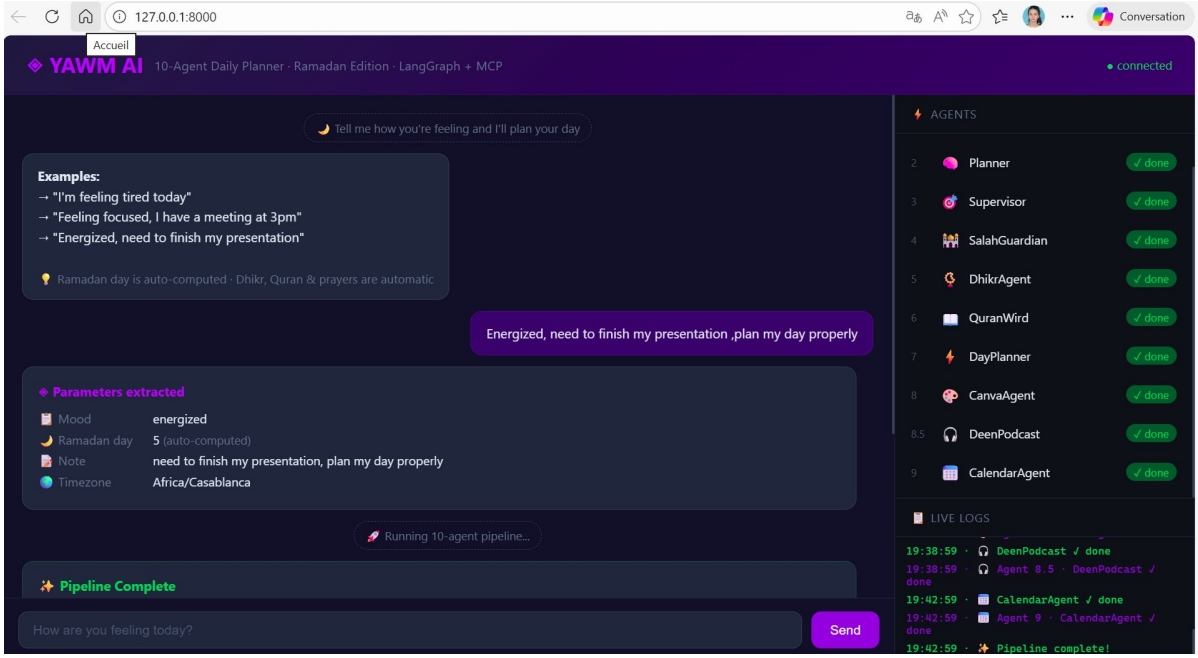


Figure 1: YAWM AI web interface: user prompt, extracted parameters, all 10 agents completing in real time, and pipeline summary.

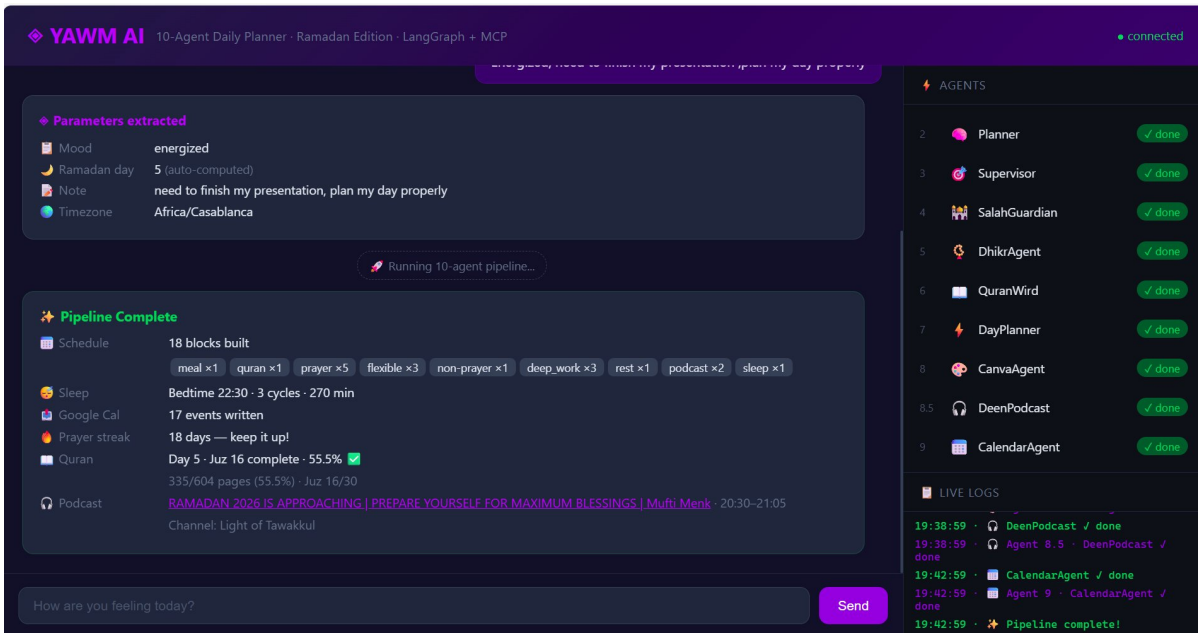


Figure 2: Pipeline Complete summary: 18 blocks, sleep window, Google Calendar events, prayer streak, Quran progress, and podcast link.

Google Calendar Output

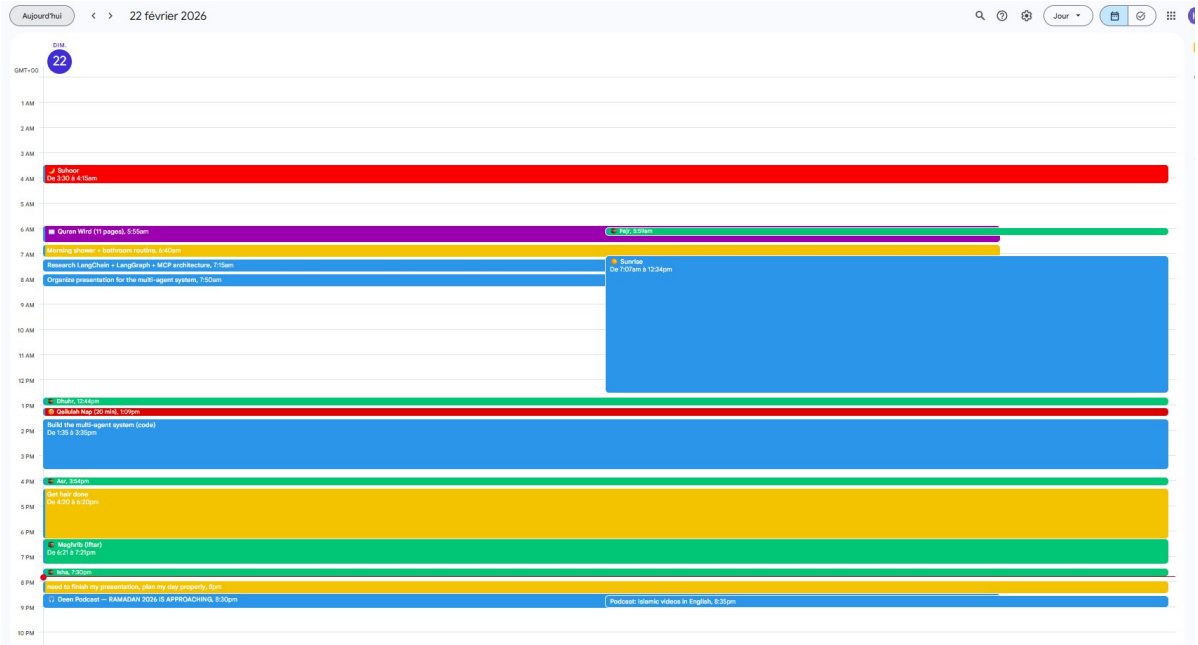


Figure 3: Google Calendar showing the full day schedule: Suhoor, Fajr, Quran Wird, deep work sessions (coding, presentation), prayers, Get hair done (2h block), Maghrib, Isha, Deen Podcast, and sleep.

Gmail Notification

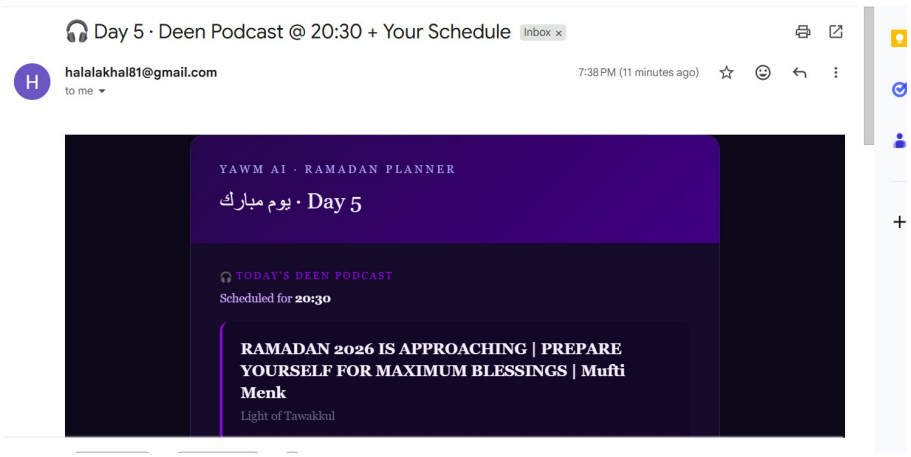


Figure 4: Gmail notification sent automatically by DeenPodcast agent: Ramadan Day 5, podcast scheduled at 20:30, with Watch on YouTube link.

Conclusion

YAWM AI demonstrates that a production multi-agent system built with LangGraph, LangChain, and MCP can automate the full daily planning loop for a Muslim during Ramadan — from task retrieval to calendar writing, with zero manual steps. Every component is real: real APIs, real LLM calls, real notifications. The system is extensible: adding a new agent, a new MCP server, or a new RAG preference requires no changes to the existing pipeline.

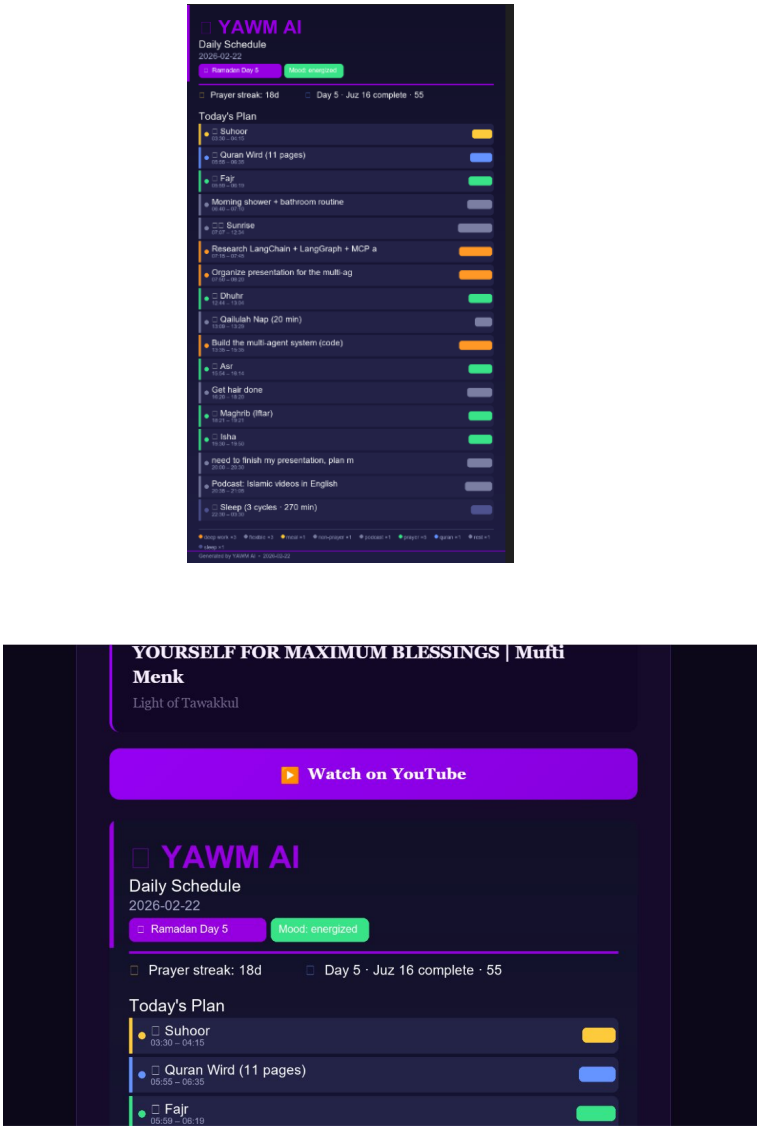
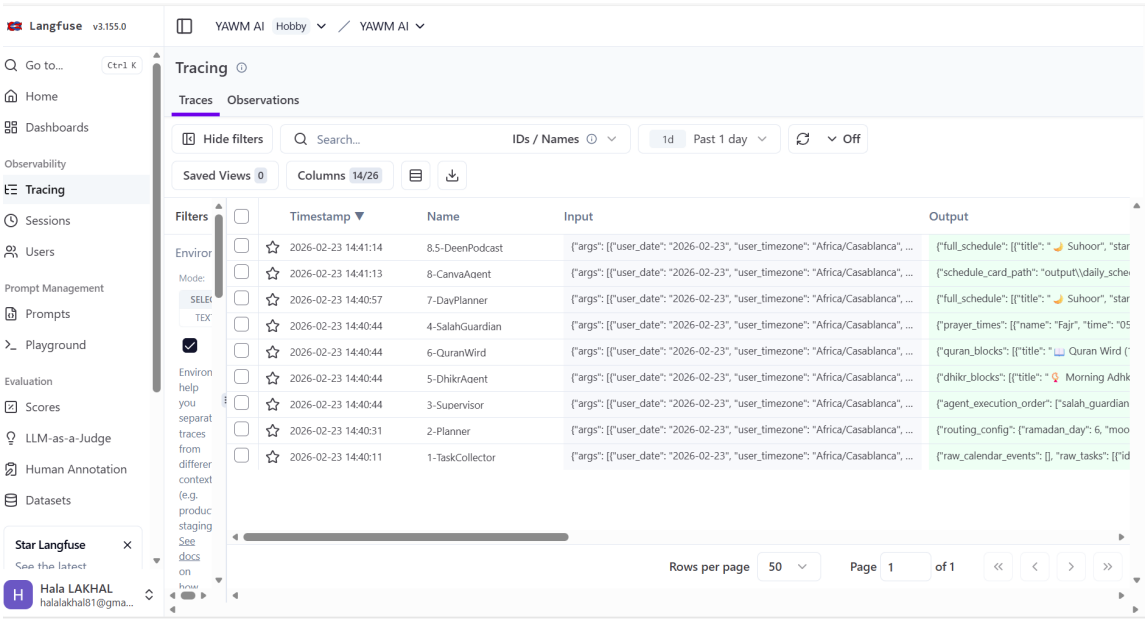


Figure 5: Email body showing the YAWM AI schedule card embedded with today's full plan, prayer streak, and Quran khatm progress.

LangFuse Tracing